

PIPELINES PROJECT PROPOSAL

Food Delivery Operations Analysis

15,000 Total Orders	30 Features per Record	13.35% Cancellation Rate	\$1.79M Total Revenue (final_amount_paid)
-------------------------------	----------------------------------	------------------------------------	--

Introduction / Background

The on-demand food delivery industry operates on razor-thin margins where operational efficiency directly determines profitability. Platforms like DoorDash, Uber Eats, and Grubhub generate millions of order records daily, each capturing the interaction between customer demand, restaurant preparation capacity, delivery partner performance, and external conditions such as traffic and weather. Despite this data richness, many mid-tier operators lack the analytical infrastructure to translate raw order logs into actionable operational insight.

This project uses this food delivery analytics dataset comprising 15,000 order records and 30 variables spanning order characteristics, delivery logistics, customer behavior, and financial outcomes. The dataset covers all 12 months of a calendar year across three city tier classifications and includes derived operational metrics such as `delivery_efficiency_score`, `delayed_delivery_flag`, and `cancellation_flag` that make it ideal for building a pipeline oriented toward operational decision-making.

The analysis is relevant to several stakeholders:

- Operations managers seeking to reduce the 13.35% cancellation rate and 9.47% delay rate
- Marketing teams evaluating the ROI of the 42.31% promo code adoption rate
- Partner success teams monitoring delivery partner efficiency, which rises steadily from 46.8 (1 year experience) to 63.3 (10 years experience)
- Finance teams tracking revenue composition across order value, delivery fees, discounts, and tips

Problem Statement

Despite the dataset containing rich operational and financial signals across 15,000 orders, there is currently no automated pipeline to load, clean, store, and surface these insights interactively. Analysts working with a flat CSV file face three compounding problems:

- **No repeatable data layer:** The raw CSV must be reloaded and re-cleaned manually for each analysis. There is no structured database with typed columns, enforced relationships, or queryable views.
- **No derived metric computation:** Key operational metrics such as on-time delivery rate, revenue margin per order, and delay severity by hour of day are not pre-calculated. They must be re-engineered from scratch each time.
- **No interactive exploration:** Static CSV analysis in Excel or Jupyter offers no cross-filtering capability. A manager cannot, for example, instantly compare cancellation rates between premium and non-premium customers across different city tiers in a single interactive view.

This project directly addresses all three gaps by building a complete ETL pipeline from raw CSV through a structured SQLite database to a Dash-powered interactive dashboard.

Objectives

- Ingest the `food_delivery_analytics_cleaned.csv` file (15,000 rows, 30 columns) into a pandas DataFrame with correct data types for all fields
- Handle the 150 missing values in `customer_rating` using median imputation, documented and auditable in the transform log
- Engineer at least four derived metrics not present in the raw file: delivery time delta (actual minus estimated), revenue margin per order, delay severity index, and normalized efficiency tier
- Design and populate a normalized SQLite database with three tables: `orders`, `city_summary`, and `partner_performance`
- Build five interactive Dash charts covering delivery performance, revenue breakdown, cancellation analysis, customer segmentation, and operational heatmap by order hour
- Ensure the pipeline runs end-to-end from a single entry-point script (`run_pipeline.py`) with no manual steps
- Document all findings in a README sufficient for a new analyst to reproduce results in under 15 minutes

Methodology

Data Source

The dataset used in this project is `food_delivery_analytics_cleaned.csv`, a pre-cleaned 15,000-record flat file containing 30 variables. No external API calls are required for data acquisition; the

pipeline begins at CSV ingestion. The file was sourced from a Kaggle public dataset archive and is fully available for educational use.

- **Key variable groups:** Order metadata (order_id, city_tier, order_hour, order_month); delivery logistics (distance_km, prep_time, delivery_time, estimated_delivery_time, efficiency_score); customer attributes (age, loyalty_score, premium_flag, customer_rating); financial fields (order_value, delivery_fee, discount_amount, tip_amount, final_amount_paid); operational flags (cancellation_flag, delayed_delivery_flag, refund_flag, promo_code_used, festival_or_weekend_flag)

Technology Stack

- **Ingestion & Transformation:** Python 3.11, pandas, numpy
- **Storage:** SQLite (file-based, zero-config), accessed via SQLAlchemy ORM
- **Visualization:** Plotly Dash, dash-bootstrap-components, plotly.express
- **Logging:** Python logging module; transform audit log written to /logs/pipeline.log
- **Reproducibility:** requirements.txt, conda environment.yml, single entry-point run_pipeline.py

ETL Architecture

LOAD	TRANSFORM	STORE	VISUALIZE
CSV → pandas	Clean + engineer features	SQLite via SQLAlchemy	Dash interactive dashboard

- **Load:** pandas reads CSV with explicit dtype mapping. All boolean flag columns cast from object to bool. order_id retained as string primary key. Null audit run immediately post-load (150 nulls found in customer_rating).
- **Transform:** Median imputation for customer_rating nulls. Four derived columns computed: delivery_delta (actual minus estimated, minutes), revenue_margin (final_amount_paid minus delivery_fee minus discount_amount), delay_severity (delivery_delta bucketed: on-time / minor / major), efficiency_tier (efficiency_score quartile label). All transforms logged with before/after row counts.
- **Store:** SQLAlchemy writes three tables to food_delivery.db. orders table holds all 15,000 rows with full column set. city_summary holds pre-aggregated metrics by city_tier. partner_performance holds efficiency and experience aggregates. Foreign key integrity validated post-load.
- **Visualize:** Dash app reads from SQLite at runtime via SQLAlchemy queries. Dropdown filters for city_tier, premium_customer_flag, and order_hour_range drive simultaneous updates across all chart panels via callback chaining.

Data Cleaning Plan

- **Missing values:** 150 nulls in customer_rating (1.0% of records) — imputed with column median (4.0). All other columns complete.

- **Type enforcement:** Six boolean flag columns stored as Python bool. Numeric columns (efficiency_score, order_value, distances) cast to float64. order_month and order_hour cast to int8 to reduce memory footprint.
- **Deduplication:** order_id verified as unique key (no duplicates expected in cleaned dataset; will confirm programmatically).
- **Outlier review:** delivery_efficiency_score min of 1.9 and delivery_time_minutes max of 180 flagged for investigation but retained (plausibly real edge cases).

Planned Dashboard Charts

- **Chart 1 — Delivery Performance KPI Cards:** On-time rate (90.53%), avg delivery time (94.1 min), avg efficiency score (59.2), and cancellation rate (13.35%) displayed as headline stats with trend indicators
- **Chart 2 — Delivery Time Distribution by City Tier:** Box plot comparing delivery_time_minutes across city tiers 1, 2, and 3; filterable by premium customer flag
- **Chart 3 — Revenue Waterfall by Component:** Stacked bar showing avg order_value, delivery_fee, discount_amount, tip_amount, and final_amount_paid per city tier
- **Chart 4 — Cancellation & Refund Heatmap:** Hour-of-day (x) vs. order_month (y) heatmap of cancellation rate; reveals peak problem windows
- **Chart 5 — Partner Efficiency by Experience:** Scatter of delivery_partner_experience_years vs. delivery_efficiency_score; overlaid with trend line showing clear positive correlation (46.8 at 1yr to 63.3 at 10yrs)

Expected Outcomes

- **Functional ETL Pipeline:** A modular Python codebase (ingest.py, transform.py, load.py, app.py) orchestrated by run_pipeline.py that processes all 15,000 records and populates the SQLite database without manual intervention.
- **Populated SQLite Database:** Three relational tables (orders, city_summary, partner_performance) with validated row counts, correct types, and documented schema in README.
- **Interactive Dash Dashboard:** Five-panel dashboard served at localhost:8050 with dropdown cross-filters for city tier, customer segment, and time of day.
- **Transform Audit Log:** A pipeline.log file that records input row counts, null counts, imputation values, derived column ranges, and load confirmation with timestamps.
- **Project Documentation:** README.md covering setup, execution, schema reference, and a data dictionary for all 30 original columns plus 4 engineered features.
- **Final Presentation:** Live pipeline execution demo showing run_pipeline.py completing in under 60 seconds followed by a guided walkthrough of all five dashboard charts.

Definition of Done: The pipeline is considered complete when run_pipeline.py executes successfully in a fresh virtual environment, produces food_delivery.db with all three populated tables, and serves a working dashboard at localhost:8050 with all five charts rendering correct data from the database.

